

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5802	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	K.Yalla reddy	Reg. No.:	192465047
Section / Batch:	2024	Date:	11-08-2026

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I K.Yalla Reddy(192465047) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Rubrics:

Criteria	Weightage	Awarded Marks	Total Marks (100)
Understanding of Concepts	25%		
Experimental Design	30%		
Use of Tools	20%		
Analysis of Results	15%		
Documentation	10%		
Total per Question	100 Marks		

— 100

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CSA5802 - DEVSECOPS ENGINEERING AND APPLICATION SECURITY

CO3- ASSESSMENT TASK 1:

SIMULATE STATIC APPLICATION SECURITY TESTING (SAST) USING SYNTHETIC DATA

NAME: YALLA REDDY.K

REG.NO:192465047

1. Abstract

Static Application Security Testing (SAST) is a shift-left security technique used to identify vulnerabilities in source code before an application is executed or deployed. This project implements an educational SAST simulator using synthetic Python source code. The synthetic application intentionally contains four security weaknesses: SQL injection, command injection, weak MD5 password hashing, and a hardcoded API key. A Python-based scanner analyzes the source code, matches predefined security patterns, assigns severity levels, calculates a risk score, and produces a JSON security report. The project also demonstrates DevSecOps integration through pre-commit and GitHub Actions configuration. The simulation detected all four intentionally introduced vulnerabilities and classified the application as HIGH risk with a score of 29.

2. Introduction

Modern software development requires security to be incorporated throughout the Software Development Life Cycle. DevSecOps extends DevOps by integrating security activities into development, testing, and deployment workflows. SAST is one of the important techniques used to achieve this objective because it analyzes application source code without requiring the program to run.

The purpose of this assessment is to simulate the behavior of a SAST tool using synthetic data. Instead of testing a real production application, a deliberately vulnerable Python program is created for educational use. A custom scanner then identifies security patterns and reports findings with severity and remediation guidance.

3. Problem Statement

Security defects that are discovered late in the development lifecycle can require significant effort to fix and may expose applications to attacks. Manual code review alone can also miss repeated security patterns.

Therefore, this project demonstrates an automated static analysis approach that can detect selected vulnerabilities early, provide actionable results, and be integrated into a DevSecOps workflow.

4. Objectives

- Create synthetic Python source code containing intentional security weaknesses.
- Develop a lightweight rule-based SAST scanner using Python.

- Detect SQL injection, command injection, weak cryptographic hashing, and hardcoded secrets.
- Assign severity levels and remediation recommendations.
- Calculate an overall risk score from detected findings.
- Generate a machine-readable JSON security report.
- Demonstrate integration with pre-commit and GitHub Actions.

5. Scope of the Project

The project focuses on educational simulation of static source-code analysis. It does not execute the vulnerable application and does not perform exploitation. The scanner uses source-text patterns to demonstrate the fundamental SAST workflow. The implementation is intentionally lightweight and is not intended to replace mature SAST platforms.

6. Tools and Technologies

Technology	Purpose
Python 3	Implementation of synthetic application and SAST scanner
Regular Expressions	Detection of hardcoded secret patterns
String Pattern Matching	Detection of selected vulnerable coding patterns
JSON	Automated scan-result reporting
YAML	Security rules and CI configuration
GitHub Actions	Continuous integration demonstration
Pre-commit	Developer-side security checking

7. System Architecture and Workflow

Source Code ☐ Static Scanner ☐ Rule Matching ☐ Finding Generation ☐ Severity Classification ☐ Risk Score ☐ JSON Report ☐ Remediation

The scanner reads the synthetic source file as text and does not execute it. Each source line is examined against security detection rules. When a rule matches, the scanner records the rule identifier, vulnerability name, severity, source line, evidence, and remediation. After scanning, a simple weighted risk score is calculated and the results are saved in JSON format.

8. Synthetic Vulnerable Application

8.1 SQL Injection

The synthetic application builds an SQL SELECT statement by concatenating username and password values into the query. This demonstrates an unsafe SQL construction pattern. The scanner identifies the pattern as SAST001 with HIGH severity. The recommended mitigation is to use parameterized queries or prepared statements.

8.2 Command Injection

The application constructs a system command using user input and passes it to a subprocess call with `shell=True`. This demonstrates a command-injection risk. The scanner identifies the pattern as SAST002 with HIGH severity. The recommended mitigation is to avoid `shell=True` where possible and validate or allowlist command arguments.

8.3 Weak Cryptographic Hash

The application uses MD5 to hash a password. MD5 is not appropriate for password storage because it is a fast general-purpose hash and is unsuitable for modern password protection. The scanner identifies SAST003 with MEDIUM severity. Recommended alternatives include Argon2, bcrypt, scrypt, or PBKDF2 with suitable configuration.

8.4 Hardcoded Secret

The synthetic application contains an API key directly in the source code. Hardcoded secrets can be exposed through source repositories, backups, or logs. The scanner identifies SAST004 with HIGH severity. The recommended mitigation is to use environment variables or a dedicated secret-management system.

9. SAST Detection Rules

Rule ID	Vulnerability	Severity	Detection Concept
SAST001	SQL Injection	HIGH	SQL query created using string concatenation
SAST002	Command Injection	HIGH	<code>shell=True</code> with command construction
SAST003	Weak Cryptographic Hash	MEDIUM	MD5 used for password hashing
SAST004	Hardcoded Secret	HIGH	API key assigned directly to a string

10. Implementation Methodology

- Read `synthetic_app.py` as plain text.
- Split the source code into individual lines.
- Apply the configured detection patterns to each line.
- Create a structured finding whenever a pattern matches.
- Record rule ID, issue, severity, line number, evidence, and remediation.
- Calculate the weighted risk score.
- Write the final results to `sast_report.json`.

11. Risk Scoring

The simulation uses the following educational scoring model: CRITICAL = 10 points, HIGH = 8 points, MEDIUM = 5 points, and LOW = 2 points.

The four detected findings produce the following score: HIGH (8) + HIGH (8) + MEDIUM (5) + HIGH (8) = 29

points. Based on the implemented threshold, the overall risk level is HIGH.

12. Scan Results

Finding	Severity	Line	Recommended Fix
SQL Injection	HIGH	7	Use parameterized SQL queries
Command Injection	HIGH	11	Avoid shell=True; validate arguments
Weak MD5 Hash	MEDIUM	15	Use Argon2/bcrypt/scrypt/PBKDF2
Hardcoded API Key	HIGH	18	Use environment variables/secret manager

Summary: Total findings = 4; HIGH = 3; MEDIUM = 1; Risk Score = 29; Risk Level = HIGH.

13. DevSecOps / Shift-Left Integration

The project includes a pre-commit configuration and a GitHub Actions workflow. The pre-commit configuration demonstrates how developers can run a security check before committing code. The GitHub Actions workflow demonstrates automatic scanning when code is pushed or when a pull request is created.

The resulting workflow is: Developer Commit → Pre-Commit SAST → Pull Request → CI SAST Scan → Findings → Remediation → Re-scan → Merge. This demonstrates the shift-left principle by moving security testing closer to the development stage.

14. Expected Repository Structure

- synthetic_app.py – synthetic vulnerable application
- scanner.py – SAST scanner implementation
- sast_rules.yml – security rule definitions
- sast_report.json – generated scan report
- findings.csv – tabular finding export
- .pre-commit-config.yaml – developer-side integration
- .github/workflows/sast.yml – CI integration
- requirements.txt – dependency declaration
- README.md – project documentation

15. Advantages

- Detects selected vulnerabilities before application execution.
- Supports early security feedback to developers.
- Can be automated in a CI/CD pipeline.
- Produces structured and repeatable results.
- Provides remediation guidance for each finding.

- Demonstrates practical application of DevSecOps shift-left security.

16. Limitations

The simulator is intentionally simple and uses pattern-based detection. Pattern matching can generate false positives and false negatives. It does not perform full abstract syntax tree analysis, data-flow analysis, taint tracking, control-flow analysis, dependency analysis, or framework-aware security analysis. Therefore, the project should be considered an educational SAST prototype rather than a production security scanner.

17. Future Enhancements

- Use Python AST analysis instead of only line-based matching.
- Add taint/data-flow analysis for user-controlled input.
- Add more rules for XSS, path traversal, insecure deserialization, and unsafe file operations.
- Scan third-party dependencies for known vulnerabilities.
- Add severity thresholds that fail the CI build for high-risk findings.
- Generate HTML/PDF dashboards and trend reports.
- Support SARIF output for integration with security platforms.

18. Conclusion

The SAST Simulation successfully demonstrates the core workflow of Static Application Security Testing using synthetic data. A deliberately vulnerable Python application was analyzed without execution, and four security weaknesses were detected: SQL injection, command injection, weak MD5 password hashing, and a hardcoded API key. The scanner classified three findings as HIGH and one as MEDIUM, producing an overall risk score of 29 and a HIGH risk classification. The addition of pre-commit and GitHub Actions configurations demonstrates how automated security checks can be integrated into a DevSecOps lifecycle. The project therefore fulfills the objective of demonstrating early, automated source-code security analysis and the shift-left security principle.

19. Appendix – Project Files

The accompanying project archive contains the editable Python, YAML, JSON, CSV, Markdown, and CI configuration files required by the assessment instructions. The vulnerable application is synthetic and is included solely for controlled educational demonstration.